Василенко Е.Р. 090303-ПИа-о24

Лабораторная работа №8
Паттерны проектирования в Python

1. Цель работы

Изучить назначение и получить практические навыки применения паттернов проектирования при разработке объектно-ориентированных приложений на Python. В ходе работы необходимо научиться распознавать типовые проблемы проектирования, выбирать подходящий паттерн для решения конкретной проблемы, реализовывать паттерны на Python, комбинировать несколько паттернов в одном приложении, а также оценивать преимущества и недостатки выбранного решения.

2. Постановка задачи

Разработать консольное приложение — систему бронирования номеров гостиницы (Вариант 5).

Предметная область: номер, гость, бронирование. Операции: бронирование номера, заселение гостя, выселение гостя, отмена бронирования, расчёт стоимости проживания.

В соответствии с вариантом необходимо применить следующие паттерны проектирования:

- Factory — создание семейства объектов «стандартный номер», «люкс», «семейный номер»;
- Strategy — взаимозаменяемые алгоритмы расчёта стоимости: обычный тариф, сезонный тариф, тариф выходного дня;
- Repository — выделение отдельного слоя доступа к данным;
- Observer — дополнительный паттерн варианта: уведомление компонентов об изменении состояния бронирования.

Дополнительные требования: использовать dataclass и Enum, выполнить рефакторинг первоначального решения, продемонстрировать работу каждого паттерна и написать не менее пяти автоматических тестов.

3. Краткие теоретические сведения

Паттерн проектирования — типовое решение распространённой проблемы проектирования программного обеспечения. Паттерн не является готовой библиотекой или фрагментом кода, который можно механически скопировать: он описывает проблему, контекст её возникновения, структуру решения, взаимодействие объектов, преимущества и ограничения.

Традиционно паттерны делятся на три группы. Порождающие паттерны (Factory, Abstract Factory, Builder, Prototype, Singleton) связаны с созданием объектов. Структурные паттерны (Adapter, Decorator, Facade, Composite, Proxy) связаны с организацией связей между объектами и классами. Поведенческие паттерны (Strategy, Observer, Command, State, Iterator, Template Method) связаны с взаимодействием объектов.

Factory отделяет создание объекта от кода, который этот объект использует: вся логика выбора конкретного класса сосредоточена в одном месте, а клиент работает с результатом фабрики. Strategy применяется, когда существует несколько взаимозаменяемых алгоритмов решения одной задачи; сервис получает стратегию извне и не изменяется при добавлении нового алгоритма. Repository скрывает детали хранения данных от бизнес-логики: сервис работает с интерфейсом и не знает, хранятся данные в памяти, файле или базе данных. Observer используется, когда изменение состояния одного объекта должно автоматически уведомлять несколько других объектов, при этом источник событий не знает деталей реализации наблюдателей.

4. Анализ первоначальной реализации (Этап 1)

Сначала функциональность была реализована без применения паттернов: тип номера, тариф и уведомления заданы цепочками if/elif, а данные хранятся в списках внутри модуля. Файл: naive/naive_hotel.py.

Файл: naive/naive_hotel.py (фрагмент)

```
def calculate_price(room_type, base_price, days, tariff):
    if room_type == "standard":
        price_per_night = base_price
    elif room_type == "suite":
        price_per_night = base_price * 2.5
    elif room_type == "family":
        price_per_night = base_price * 1.6
    else:
        raise ValueError("Неизвестный тип номера")
```

```
if tariff == "regular":
    total = price_per_night * days
elif tariff == "seasonal":
    total = price_per_night * days * 1.3
elif tariff == "weekend":
    total = price_per_night * days * (0.85 if days >= 2 else 1.0)
else:
    raise ValueError("Неизвестный тариф")

return round(total, 2)
```

Проблемы первоначального решения:

• чтобы добавить новый тип номера, нужно изменить функцию calculate_price и добавить ветку в add_room;

• новый тариф также требует правки calculate_price, то есть уже работающий код постоянно изменяется;

• уведомления вызываются жёстко внутри book_room, поэтому новый получатель события требует изменения сервиса;

• хранение данных (списки словарей) перемешано с бизнес-логикой — заменить его на файл или базу данных без правки сервиса невозможно;

• расчёт стоимости нельзя протестировать отдельно от консольного меню.

Таким образом, наивная реализация нарушает принципы единственной ответственности (SRP) и открытости/закрытости (OCP), а её расширение каждый раз затрагивает существующий рабочий код. Это и стало причиной выбора паттернов проектирования.

5. Выбор паттернов

Для каждой выявленной проблемы выбран паттерн, который решает её без усложнения проекта.

Проблема	**Паттерн**	**Причина выбора**	**Результат**
Создание номеров разных типов сосредоточено в if/elif	Factory	Типы номеров образуют семейство классов с общим интерфейсом	RoomFactory.create()
Расчёт стоимости «зашит» в условную логику	Strategy	Существует несколько взаимозаменяемых алгоритмов расчёта	BookingService.set_strategy()

Проблема	Паттерн	Причина выбора	Результат
Данные хранятся внутри сервиса	Repository	Способ хранения должен быть заменяемым	InMemoryRepository, зависимость от интерфейса
Уведомления вызываются жёстко	Observer	Несколько независимых получателей одного события	attach(), EventLogger, GuestNotifier

Factory. Проблема: код создания объекта и код его использования были перемешаны. Паттерн: RoomFactory. Причина: типы номеров различаются характеристиками (коэффициент цены, вместимость), но имеют общую структуру, поэтому их удобно создавать в одном месте. Структура: базовый класс Room и три наследника, фабрика хранит словарь «тип — класс» и создаёт нужный объект по значению перечисления RoomType. Результат: при добавлении нового типа номера достаточно зарегистрировать класс в фабрике, клиентский код не меняется.

Strategy. Проблема: расчёт стоимости был реализован условной конструкцией, которая изменялась при добавлении тарифа. Паттерн: PricingStrategy с реализациями RegularTariff, SeasonalTariff и WeekendTariff. Причина: алгоритмы взаимозаменяемы и решают одну задачу — вычисление стоимости проживания. Структура: интерфейс с методом calculate(room, days), сервис хранит ссылку на стратегию и получает её через конструктор или метод set_strategy(). Результат: тариф меняется во время работы программы, код сервиса при этом не изменяется.

Repository. Проблема: коллекции объектов находились внутри сервиса, поэтому замена хранилища затрагивала бизнес-логику. Паттерн: Repository. Причина: сервис должен зависеть от абстракции, а не от конкретного способа хранения. Структура: Protocol Repository с методами add, get, get_all, update, delete и реализация InMemoryRepository на словаре. Результат: сервис не знает, где хранятся данные, а реализацию можно заменить без его изменения.

Observer. Проблема: уведомления вызывались напрямую, и каждый новый получатель требовал правки метода бронирования. Паттерн: Observer. Причина: об изменении состояния бронирования должны узнавать несколько независимых компонентов — журнал событий и уведомление гостя. Структура: интерфейс Observer с методом update(event), список наблюдателей и методы attach/detach в сервисе, событие Event с типом и сообщением. Результат: новый наблюдатель подключается одной строкой, сервис о нём ничего не знает.

6. Диаграмма классов

Итоговая архитектура приложения приведена на рисунке 8.1. Пунктирная линия с полым треугольником обозначает реализацию интерфейса, сплошная линия с полым треугольником — наследование, пунктирная стрелка — зависимость.

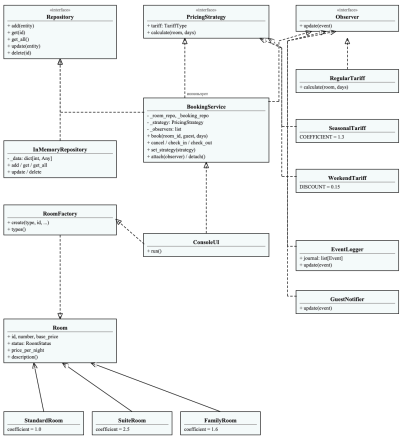


Рисунок 8.1 – UML-диаграмма классов

7. Реализация

7.1. Структура проекта

```
lab_8/
├── main.py
├── models/
│   ├── enums.py
│   └── entities.py
├── interfaces/
│   ├── repository.py
│   ├── pricing.py
│   └── observer.py
├── repositories/
│   └── memory_repository.py
├── factories/
│   └── room_factory.py
├── strategies/
│   └── pricing.py
├── observers/
│   └── observers.py
├── services/
│   └── booking_service.py
├── ui/
│   └── console.py
├── naive/
│   └── naive_hotel.py
└── tests/
    └── test_hotel.py
```

7.2. Модели и перечисления

Файл: models/enums.py

```
from enum import Enum


class RoomType(Enum):
    STANDARD = "стандартный"
    SUITE = "люкс"
    FAMILY = "семейный"


class RoomStatus(Enum):
    FREE = "свободен"
    BOOKED = "забронирован"
    OCCUPIED = "занят"
    MAINTENANCE = "на обслуживании"


class BookingStatus(Enum):
    CREATED = "создано"
    ACTIVE = "гость проживает"
    COMPLETED = "завершено"
    CANCELLED = "отменено"


class TariffType(Enum):
    REGULAR = "обычный тариф"
    SEASONAL = "сезонный тариф"
    WEEKEND = "тариф выходного дня"


class EventType(Enum):
    BOOKING_CREATED = "бронирование создано"
    BOOKING_CANCELLED = "бронирование отменено"
    CHECK_IN = "заселение"
    CHECK_OUT = "выселение"
```

Файл: models/entities.py

```
from dataclasses import dataclass
from typing import ClassVar, Optional
from .enums import BookingStatus, EventType, RoomStatus, RoomType, TariffType


# Базовая сущность "номер". Конкретные типы номеров задают свои
# характеристики через ClassVar, поэтому конструктор у всех одинаковый.
@dataclass
class Room:
    id: int
    number: str
    base_price: float
    status: RoomStatus = RoomStatus.FREE

    room_type: ClassVar[RoomType] = RoomType.STANDARD
    capacity: ClassVar[int] = 2
    coefficient: ClassVar[float] = 1.0

    def __post_init__(self):
        if self.base_price <= 0:
            raise ValueError("Базовая цена номера должна быть больше 0")

    @property
    def price_per_night(self) -> float:
        return round(self.base_price * self.coefficient, 2)

    def description(self) -> str:
        return (
            f"№{self.number} ({self.room_type.value}, до {self.capacity}
гостей), "
            f"{self.price_per_night:.2f} руб./ночь, статус: {self.status.value}"
        )


@dataclass
class StandardRoom(Room):
    room_type: ClassVar[RoomType] = RoomType.STANDARD
    capacity: ClassVar[int] = 2
    coefficient: ClassVar[float] = 1.0


@dataclass
class SuiteRoom(Room):
    room_type: ClassVar[RoomType] = RoomType.SUITE
    capacity: ClassVar[int] = 2
    coefficient: ClassVar[float] = 2.5


@dataclass
class FamilyRoom(Room):
    room_type: ClassVar[RoomType] = RoomType.FAMILY
    capacity: ClassVar[int] = 4
    coefficient: ClassVar[float] = 1.6


@dataclass
class Booking:
    id: int
    room_id: int
    room_number: str
    guest_name: str
    days: int
    tariff: TariffType
    total_cost: float
    status: BookingStatus = BookingStatus.CREATED

    def description(self) -> str:
        return (
            f"№{self.id}: гость {self.guest_name}, номер {self.room_number}, "
```

```
            f"{self.days} дн., {self.tariff.value}, {self.total_cost:.2f} руб.,
"
            f"статус: {self.status.value}"
        )


# Событие, которое сервис рассылает наблюдателям
@dataclass
class Event:
    type: EventType
    message: str
```

7.3. Factory

Файл: factories/room_factory.py

```
from typing import Dict, List, Type
from models.entities import FamilyRoom, Room, StandardRoom, SuiteRoom
from models.enums import RoomType


# Factory: единственное место, где по типу создаётся нужный класс номера.
# Клиентский код (UI, тесты) не знает конкретных классов.
class RoomFactory:
    _ROOMS: Dict[RoomType, Type[Room]] = {
        RoomType.STANDARD: StandardRoom,
        RoomType.SUITE: SuiteRoom,
        RoomType.FAMILY: FamilyRoom,
    }

    @staticmethod
    def create(room_type: RoomType, room_id: int, number: str, base_price:
float) -> Room:
        room_class = RoomFactory._ROOMS.get(room_type)
        if room_class is None:
            raise ValueError(f"Неизвестный тип номера: {room_type}")
        return room_class(id=room_id, number=number, base_price=base_price)

    @staticmethod
    def types() -> List[RoomType]:
        return list(RoomFactory._ROOMS.keys())
```

Фабрика используется и в основном приложении: в main.py номера создаются вызовом RoomFactory.create(RoomType.STANDARD, 1, "101", 2500), а также в консольном интерфейсе при добавлении номера.

7.4. Strategy

Файл: interfaces/pricing.py

```
from typing import Protocol
from models.entities import Room
from models.enums import TariffType


# Общий интерфейс стратегии расчёта стоимости проживания.
class PricingStrategy(Protocol):
    tariff: TariffType

    def calculate(self, room: Room, days: int) -> float: ...
```

Файл: strategies/pricing.py

```
from interfaces.pricing import PricingStrategy
from models.entities import Room
from models.enums import TariffType


# Обычный тариф: стоимость = цена за ночь * количество ночей.
```

```
class RegularTariff:
    tariff = TariffType.REGULAR

    def calculate(self, room: Room, days: int) -> float:
        return round(room.price_per_night * days, 2)


# Сезонный тариф: наценка 30% в высокий сезон.
class SeasonalTariff:
    tariff = TariffType.SEASONAL
    COEFFICIENT = 1.3

    def calculate(self, room: Room, days: int) -> float:
        return round(room.price_per_night * days * self.COEFFICIENT, 2)


# Тариф выходного дня: скидка 15% при проживании от двух ночей.
class WeekendTariff:
    tariff = TariffType.WEEKEND
    DISCOUNT = 0.15

    def calculate(self, room: Room, days: int) -> float:
        cost = room.price_per_night * days
        if days >= 2:
            cost *= 1 - self.DISCOUNT
        return round(cost, 2)
```

7.5. Repository

Файл: interfaces/repository.py

```
from typing import Any, List, Protocol, Optional


# Репозиторий скрывает способ хранения данных от бизнес-логики.
# Сервис работает только с этим интерфейсом (DIP).
class Repository(Protocol):
    def add(self, entity: Any) -> None: ...

    def get(self, entity_id: int) -> Optional[Any]: ...

    def get_all(self) -> List[Any]: ...

    def update(self, entity: Any) -> None: ...

    def delete(self, entity_id: int) -> None: ...
```

Файл: repositories/memory_repository.py

```
from typing import Any, List, Optional
from interfaces.repository import Repository


# Хранилище в оперативной памяти. Сервис о нём ничего не знает.
class InMemoryRepository(Repository):
    def __init__(self):
        self._data: dict[int, Any] = {}

    def add(self, entity: Any) -> None:
        if entity.id in self._data:
            raise ValueError(f"Объект с ID {entity.id} уже существует")
        self._data[entity.id] = entity

    def get(self, entity_id: int) -> Optional[Any]:
        return self._data.get(entity_id)

    def get_all(self) -> List[Any]:
        return list(self._data.values())

    def update(self, entity: Any) -> None:
        if entity.id not in self._data:
            raise ValueError(f"Объект с ID {entity.id} не найден")
        self._data[entity.id] = entity
```

```
    def delete(self, entity_id: int) -> None:
        self._data.pop(entity_id, None)
```

7.6. Observer

Файл: interfaces/observer.py

```
from typing import Protocol
from models.entities import Event


# Наблюдатель получает событие и сам решает, что с ним делать.
class Observer(Protocol):
    def update(self, event: Event) -> None: ...
```

Файл: observers/observers.py

```
from typing import List
from interfaces.observer import Observer
from models.entities import Event


# Наблюдатель 1: ведёт журнал всех событий гостиницы.
class EventLogger(Observer):
    def __init__(self):
        self.journal: List[Event] = []

    def update(self, event: Event) -> None:
        self.journal.append(event)
        print(f"[ЖУРНАЛ] {event.type.value}: {event.message}")

    def show(self) -> str:
        if not self.journal:
            return "Журнал пуст."
        lines = ["--- ЖУРНАЛ СОБЫТИЙ ---"]
        for index, event in enumerate(self.journal, start=1):
            lines.append(f"{index}. [{event.type.value}] {event.message}")
        return "\n".join(lines)


# Наблюдатель 2: уведомляет гостя об изменении бронирования.
class GuestNotifier(Observer):
    def update(self, event: Event) -> None:
        print(f"[УВЕДОМЛЕНИЕ] {event.message}")
```

7.7. Сервис бронирования

Сервис не зависит от пользовательского интерфейса, от конкретного хранилища и от конкретного алгоритма расчёта: все три зависимости передаются через конструктор и описаны интерфейсами.

Файл: services/booking_service.py

```
from typing import List
from interfaces.observer import Observer
from interfaces.pricing import PricingStrategy
from interfaces.repository import Repository
from models.entities import Booking, Event, Room
from models.enums import BookingStatus, EventType, RoomStatus, TariffType


# Бизнес-логика бронирования. Не зависит от UI, от способа хранения
# (Repository) и от конкретного алгоритма расчёта цены (Strategy).
class BookingService:
    def __init__(self, room_repo: Repository, booking_repo: Repository,
strategy: PricingStrategy):
        self._room_repo = room_repo
        self._booking_repo = booking_repo
        self._strategy = strategy
        self._observers: List[Observer] = []
```

```
    # --- Observer: подписка на события ---
    def attach(self, observer: Observer) -> None:
        self._observers.append(observer)

    def detach(self, observer: Observer) -> None:
        if observer in self._observers:
            self._observers.remove(observer)

    def _notify(self, event_type: EventType, message: str) -> None:
        event = Event(type=event_type, message=message)
        for observer in self._observers:
            observer.update(event)

    # --- Strategy: тариф меняется без изменения сервиса ---
    def set_strategy(self, strategy: PricingStrategy) -> None:
        self._strategy = strategy

    @property
    def tariff(self) -> TariffType:
        return self._strategy.tariff

    # --- Операции ---
    def calculate_price(self, room_id: int, days: int) -> float:
        room = self._get_room(room_id)
        self._validate_days(days)
        return self._strategy.calculate(room, days)

    def book(self, room_id: int, guest_name: str, days: int) -> Booking:
        room = self._get_room(room_id)
        self._validate_days(days)

        if room.status != RoomStatus.FREE:
            raise ValueError(f"Номер {room.number} недоступен (статус:
{room.status.value})")

        total_cost = self._strategy.calculate(room, days)

        room.status = RoomStatus.BOOKED
        self._room_repo.update(room)

        booking = Booking(
            id=self._next_booking_id(),
            room_id=room.id,
            room_number=room.number,
            guest_name=guest_name,
            days=days,
            tariff=self._strategy.tariff,
            total_cost=total_cost,
        )
        self._booking_repo.add(booking)

        self._notify(
            EventType.BOOKING_CREATED,
            f"Гость {guest_name} забронировал номер {room.number} на {days} дн.
"
            f"({self._strategy.tariff.value}), стоимость {total_cost:.2f} руб.",
        )
        return booking

    def cancel(self, booking_id: int) -> None:
        booking = self._get_booking(booking_id)
        if booking.status != BookingStatus.CREATED:
            raise ValueError(
                f"Нельзя отменить бронирование №{booking.id} (статус:
{booking.status.value})"
            )

        room = self._get_room(booking.room_id)
        room.status = RoomStatus.FREE
        self._room_repo.update(room)
```

```
        booking.status = BookingStatus.CANCELLED
        self._booking_repo.update(booking)

        self._notify(
            EventType.BOOKING_CANCELLED,
            f"Бронирование №{booking.id} гостя {booking.guest_name} отменено,
номер {room.number} свободен.",
        )


    ...  # методы check_in, check_out и вспомогательные методы
```

Полный исходный код остальных методов (заселение, выселение, проверка статусов, генерация идентификатора) находится в файле services/ booking_service.py.

7.8. Сборка приложения

Файл: main.py

```
from factories.room_factory import RoomFactory
from models.enums import RoomType
from observers.observers import EventLogger, GuestNotifier
from repositories.memory_repository import InMemoryRepository
from services.booking_service import BookingService
from strategies.pricing import RegularTariff
from ui.console import ConsoleUI


def build_service() -> tuple[BookingService, EventLogger]:
    # Repository скрывает хранение: сервис получает только интерфейс
    room_repo = InMemoryRepository()
    booking_repo = InMemoryRepository()

    # Strategy передаётся в сервис извне и может быть заменена в любой момент
    service = BookingService(room_repo, booking_repo, RegularTariff())

    # Observer: журнал и уведомления гостю подписываются на события сервиса
    event_logger = EventLogger()
    service.attach(event_logger)
    service.attach(GuestNotifier())

    # Номера создаются через Factory, а не через конструкторы напрямую
    service.add_room(RoomFactory.create(RoomType.STANDARD, 1, "101", 2500))
    service.add_room(RoomFactory.create(RoomType.SUITE, 2, "201", 2500))
    service.add_room(RoomFactory.create(RoomType.FAMILY, 3, "301", 2500))

    return service, event_logger


def main():
    service, event_logger = build_service()
    ui = ConsoleUI(service, event_logger)
    ui.run()


if __name__ == "__main__":
    main()
```

Консольный интерфейс отвечает только за ввод и вывод. Переключение тарифа выполняется вызовом set_strategy(), поэтому добавление нового тарифа не требует изменения ни сервиса, ни интерфейса — достаточно добавить класс стратегии и пункт меню.

Файл: ui/console.py (фрагмент)

```
def _change_tariff(self):
    print("\nДоступные тарифы:")
    for key, strategy in TARIFFS.items():
        print(f"  {key}. {strategy.tariff.value}")
    tariff_key = input("Выберите тариф: ").strip()
    strategy_class = TARIFFS.get(tariff_key)
    if strategy_class is None:
        raise ValueError("Неизвестный тариф")
    self.booking_service.set_strategy(strategy_class())
    print(f"Тариф изменён: {self.booking_service.tariff.value}")

def _show_bookings(self):
```

8. Результаты работы

Тест 1. Запуск программы и вывод списка номеров, созданных через RoomFactory. В списке представлены все три типа номеров с разными коэффициентами стоимости (рисунок 8.2).

```
lab_8 — python3 main.py

--- МЕНЮ ГОСТИНИЦЫ ---
1. Показать номера
2. Добавить номер
3. Рассчитать стоимость проживания
4. Забронировать номер
5. Заселить гостя
6. Выселить гостя
7. Отменить бронирование
8. Сменить тариф (сейчас: обычный тариф)
9. Показать бронирования
10. Показать журнал событий
0. Выход
Выберите действие: 1
    руб./ночь, статус: свободен
Номера гостиницы:
  ID 1: №101 (стандартный, до 2 гостей), 2500.00 руб./ночь, статус: свободен
  ID 2: №201 (люкс, до 2 гостей), 6250.00 руб./ночь, статус: свободен
  ID 3: №301 (семейный, до 4 гостей), 4000.00 руб./ночь, статус: свободен
Выберите действие:
Работа завершена.
```

Рисунок 8.2 – Запуск программы и список номеров

Тест 2. Бронирование номера. Сервис проверяет доступность номера, рассчитывает стоимость по текущей стратегии, сохраняет бронирование в репозиторий и уведомляет наблюдателей: запись появляется в журнале и выводится уведомление гостю (рисунок 8.3).

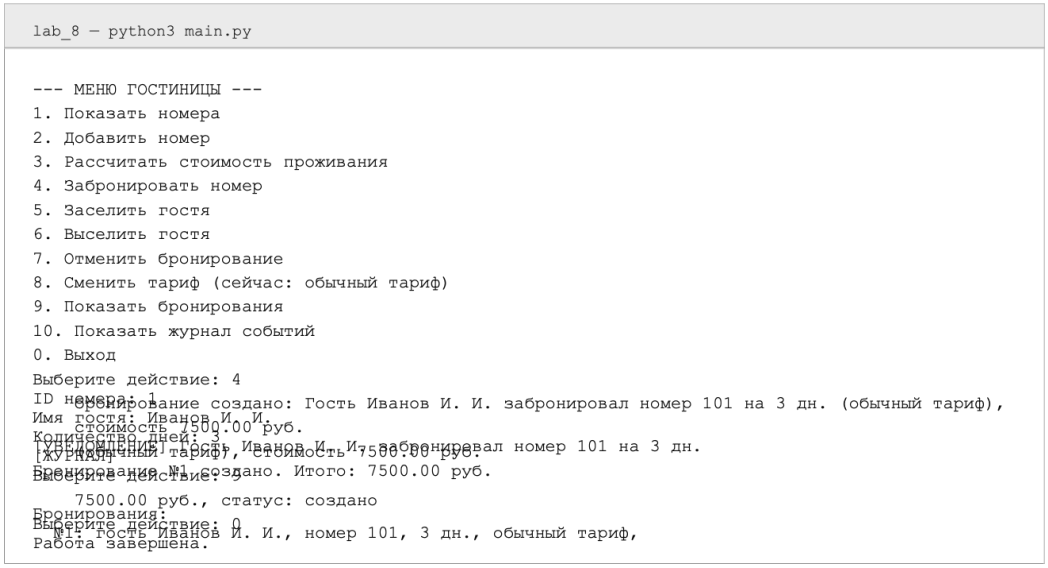


Рисунок 8.3 – Бронирование номера и работа Observer

Тест 3. Переключение тарифа. Для одного и того же номера и срока стоимость по обычному тарифу составляет 18 750 руб., после смены стратегии на сезонную — 24 375 руб. Код сервиса при этом не изменялся (рисунок 8.4).

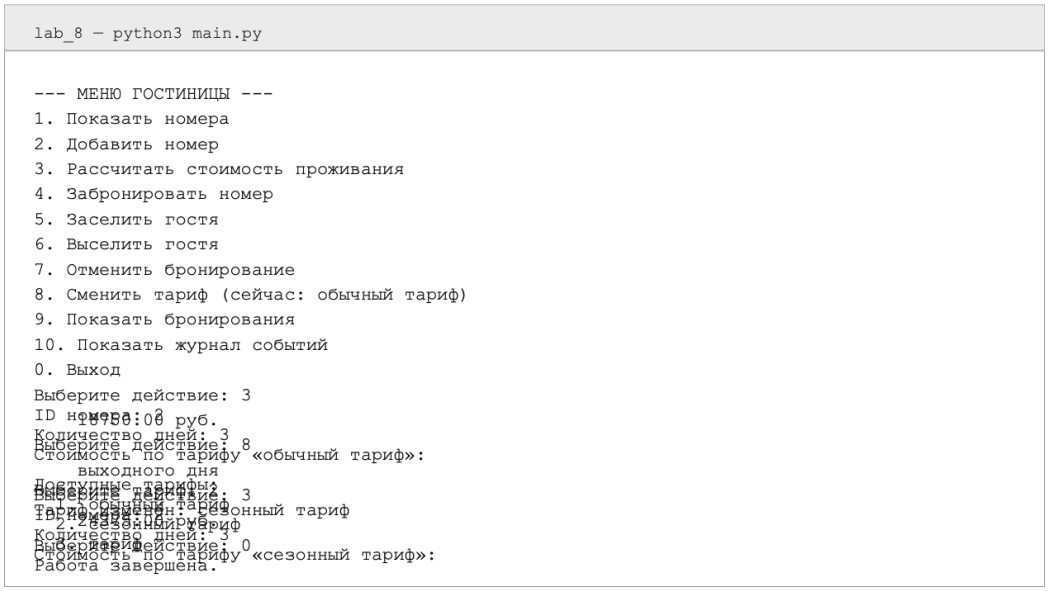


Рисунок 8.4 – Смена тарифа и расчёт стоимости

Тест 4. Заселение гостя и просмотр журнала событий. После заселения статус номера изменяется на «занят», а оба наблюдателя получают событие CHECK_IN (рисунок 8.5).

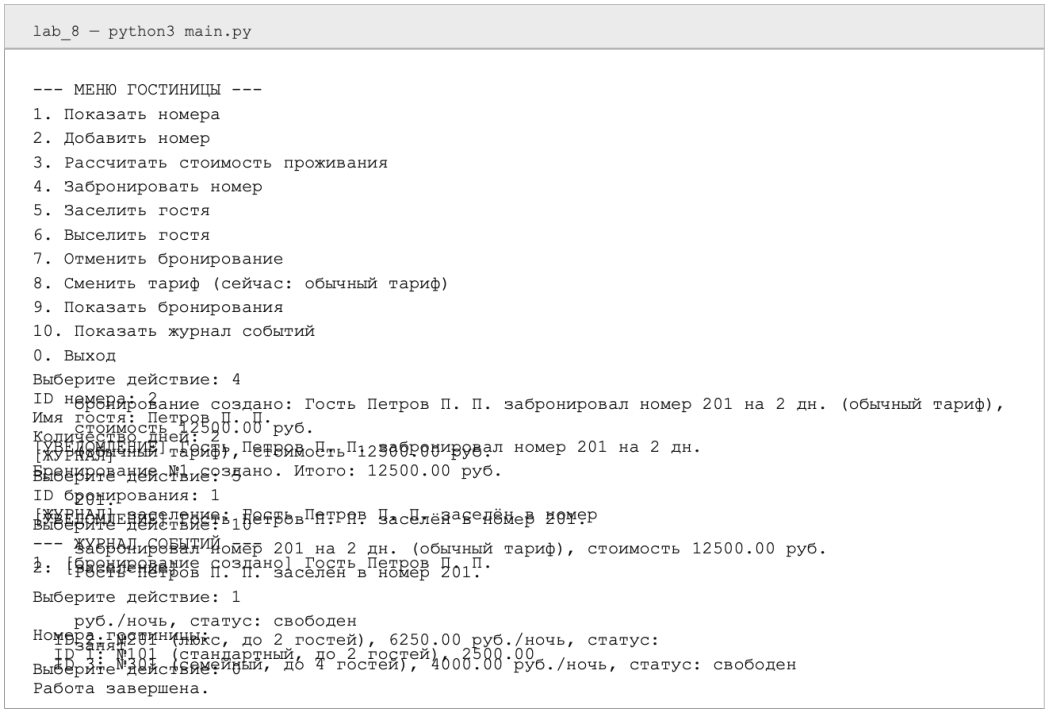


Рисунок 8.5 – Заселение гостя и журнал событий

Тест 5. Автоматические тесты. Проверяются фабрика, стратегии, репозиторий, основная операция бронирования и работа наблюдателей — всего 13 тестов (рисунок 8.6).



Рисунок 8.6 – Результаты автоматических тестов

9. Сравнение решений

Критерий	До рефакторинга	После
Связанность	Сервис, хранение и уведомления в одном модуле	Слои связаны через интерфейсы
Расширяемость	Новый тип или тариф требует правки calculate_price	Достаточно добавить класс и зарегистрировать его
Тестируемость	Логика не отделена от меню	13 автоматических тестов
Создание объектов	if/elif по строковому типу	RoomFactory
Выбор алгоритма	if/elif по названию тарифа	Strategy и set_strategy()
Хранение данных	списки внутри модуля	Repository (InMemoryRepository)
Уведомления	жёсткие вызовы print	Observer (журнал, уведомление гостю)

10. Вывод

В ходе выполнения лабораторной работы были изучены паттерны проектирования и получены практические навыки их применения. На первом этапе была реализована наивная версия системы бронирования гостиницы, в которой создание объектов, расчёт стоимости, хранение данных и уведомления были реализованы условными конструкциями внутри одного модуля. Анализ показал, что такое решение плохо расширяется и не поддаётся тестированию.

На втором этапе приложение было переработано: создание номеров вынесено в Factory, расчёт стоимости — в семейство стратегий, хранение — в Repository, а уведомления — в Observer. Бизнес-логика перестала зависеть от интерфейса, способа хранения и конкретного алгоритма расчёта, что подтверждается автоматическими тестами и работой программы. Главный вывод работы: паттерн проектирования — не цель программы, а средство решения конкретной архитектурной проблемы.